ПРАКТИЧЕСКИЙ ПРОЕКТ • ТЕХНИЧЕСКОЕ ЗАДАНИЕ

TaskFlow

От первого Flask endpoint до production deployment

Пошаговый инженерный маршрут для начинающего Python Backend Developer. Отдельный React frontend, Flask REST API, PostgreSQL, Redis, Celery, Docker, мониторинг, тесты и CI/CD.

Формат	Уровень	Результат
Учебный проект + критерии приёмки	Beginner → Junior+	Работающее приложение, GitHub repo и production URL

Октябрь 2026

Правило: сначала понять задачу и технологию, затем реализовать. Не копировать готовые решения без объяснения.

ЭТАП 00

Как пользоваться этим документом

Цель: Пройти все этапы в указанном порядке и собрать работающую систему.

• Для каждого этапа: прочитай требования → изучи термины → создай отдельную Git-ветку → реализуй → напиши тесты → открой Pull Request → продемонстрируй результат.

• Не переходи к следующему этапу, пока не выполнены критерии готовности предыдущего.

• Веди README: команды запуска, принятые решения, известные ограничения и скриншоты.

• Основной объём — обязательный. Пометки «Advanced» выполняются после базового релиза.

• Вместе с ТЗ выдаются макеты экранов «TaskFlow — UI дизайн». Для каждого экрана в них указаны endpoints и реакция интерфейса на ошибки — используй их как контракт между frontend и backend. Демо-данные на макетах совпадают с seed-данными этапа 04.

Что в итоге должно работать

Пользователь регистрируется, создаёт workspace, добавляет участников, создаёт проекты и задачи, назначает исполнителей, оставляет комментарии и прикрепляет файлы. Система отправляет уведомления, проверяет дедлайны по расписанию, ведёт историю изменений, показывает статистику и предоставляет мониторинг для администратора.

Содержание и маршрут

Блок	Этапы
Фундамент	01 окружение · 02 Git и Docker · 03 структура, тесты и CI · 04 БД
API и бизнес-логика	05 REST · 06 валидация · 07 auth/RBAC · 08 tasks
Интеграции	09 Redis · 10 Celery · 11 cron · 12 файлы
Качество	13 frontend · 14 тесты · 15 observability · 16 безопасность
Релиз	17 CI/CD · 18 production · 19 финальная приёмка · 20 advanced
Процесс	21 code review — действует с первого Pull Request (этап 02)

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: HTTP, API, REST, JSON, client-server, backend, frontend, CRUD, stateless, deployment.

Контрольные вопросы: Чем API отличается от UI? Почему frontend не должен подключаться напрямую к PostgreSQL?

ЭТАП 01

Подготовка компьютера

Цель: Подготовить воспроизводимое окружение разработки.

• Установи Python 3.13+, Git, Docker Engine / Docker Desktop, Docker Compose plugin, IDE, curl или Postman, PostgreSQL client.

• Создай GitHub account, SSH-ключ или безопасную HTTPS-аутентификацию. Проверь команды python --version, git --version, docker compose version.

• Создай каталог taskflow, README.md и .gitignore. Опиши требования к окружению и команды запуска.

• Создай виртуальное окружение, pyproject.toml, зафиксируй зависимости. Выбери один package manager и объясни выбор.

Критерий готовности: Новый разработчик может клонировать репозиторий и повторить шаги установки по README.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Python virtual environment, pip/uv, dependency locking, semantic versioning, Git, SSH key, .gitignore.

Контрольные вопросы: Зачем виртуальное окружение? Почему нельзя хранить .env с паролями в Git?

ЭТАП 02

Git, репозиторий и первый Docker

Цель: Запустить минимальное приложение в контейнере.

• Создай два независимых каталога frontend/ и backend/ внутри одного учебного репозитория (допустимы и два репозитория). Между ними только HTTP API.

• Создай Flask app factory, endpoint GET /api/v1/health, возвращающий JSON {"status":"ok"}.

• Создай Dockerfile backend (не root-пользователь, .dockerignore), compose.yaml и .env.example.

• Запусти API через Gunicorn в контейнере. Настрой port mapping и hot reload только для development.

• Сделай первый feature branch, PR и merge. Добавь инструкции docker compose up --build и curl /api/v1/health.

• С этого PR действуют правила code review из этапа 21.

```
git checkout -b feat/bootstrap
docker compose up --build -d
curl http://localhost:8000/api/v1/health
docker compose logs -f api
```

Критерий готовности: Одна команда поднимает API, health endpoint возвращает HTTP 200; секреты не попали в Git.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Docker image vs container, Dockerfile, Compose, port, volume, network, Gunicorn, WSGI, Git branch, Pull Request.

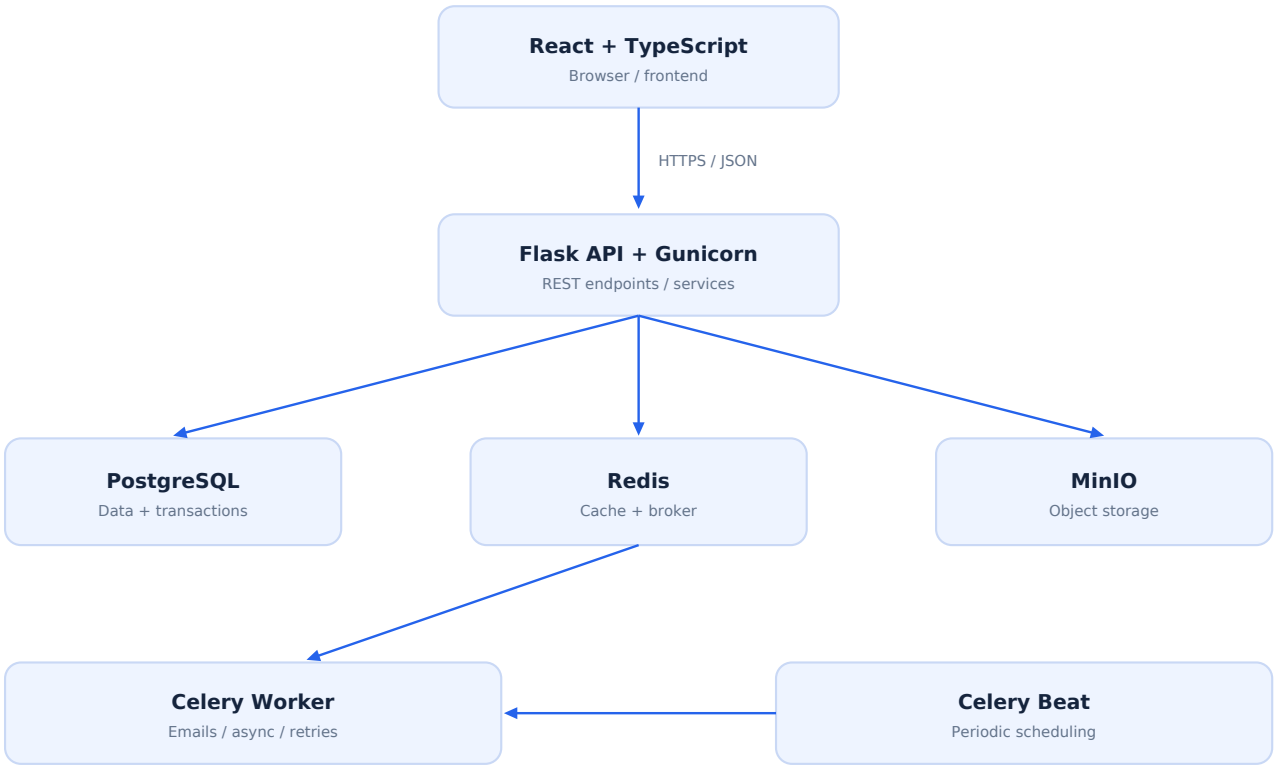
Контрольные вопросы: Почему localhost внутри контейнера не равен localhost компьютера?

ЭТАП 03

Архитектура, структура и первые тесты

Цель: Разделить HTTP, бизнес-логику, данные и внешние интеграции — и с первого дня проверять код автоматически.

Схема взаимодействия компонентов



Flask — единый backend API. PostgreSQL хранит постоянные данные. Redis используется для кэша и как брокер Celery. Celery Worker выполняет фоновые задачи; Celery Beat планирует периодические задания. MinIO хранит вложения. Метрики, логи и ошибки собираются отдельно.

Рекомендуемая структура репозитория

```
taskflow/
  backend/
    app/
      __init__.py          # create_app()
      config.py            # settings / env
      api/v1/              # HTTP routes / blueprints
      schemas/             # Pydantic request/response
      models/              # SQLAlchemy entities
      repositories/        # database access
      services/            # business rules
      tasks/               # Celery jobs
      integrations/        # email, S3, Redis
      core/                # auth, errors, logging
    migrations/            # Alembic
    tests/conftest.py      # fixtures: app, client, db
    tests/unit/
    tests/integration/
    pyproject.toml
    Dockerfile
  frontend/                # React + TypeScript
  infra/
    prometheus/
    grafana/
    loki/                  # advanced (этап 15)
    nginx/
  .github/workflows/       # ci.yml — с этапа 03
  compose.yaml
  compose.prod.yaml
  .env.example
  README.md
```

• Запрещено помещать SQL-запросы и всю бизнес-логику прямо в route handlers. Route принимает запрос, валидирует и вызывает service.

• Repository инкапсулирует работу с БД; service реализует правила предметной области; schema определяет API-контракт.

Тесты и CI с первого дня

• Подключи pytest: фикстуры app и client через create_app() с тестовой конфигурацией. Первый тест — GET /api/v1/health возвращает 200 и {"status":"ok"}.

• Добавь Ruff (lint + format) и одну команду запуска всех проверок; опиши её в README.

• Минимальный CI: GitHub Actions на каждый PR запускает Ruff и pytest. Включи protected branch — merge в main только при зелёных проверках.

Критерий готовности: Структура создана, зависимости между слоями описаны в README; health endpoint использует app factory; PR с падающим тестом не проходит CI.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Separation of concerns, layered architecture, dependency injection, Flask Blueprint, app factory, repository pattern, service layer, pytest fixture, GitHub Actions.

Контрольные вопросы: Зачем отделять route от service? Что должно быть в repository? Почему CI выгоднее включить в начале проекта, а не перед релизом?

ЭТАП 04

PostgreSQL и модель данных

Цель: Спроектировать реляционную схему и миграции.

Сущность	Обязательные поля / ограничения
users	id UUID PK, email UNIQUE, password_hash, display_name, created_at, is_active
workspaces	id, name, owner_id FK → users, created_at
memberships	id, workspace_id FK, user_id FK, role; UNIQUE(workspace_id,user_id)
projects	id, workspace_id FK, name, description, created_at
tasks	id, project_id FK, title, description, status, priority, assignee_id nullable FK, due_at nullable, version, created_at, updated_at
comments	id, task_id FK, author_id FK, body, created_at
attachments	id, task_id FK, uploader_id FK, object_key, filename, size_bytes, content_type
notifications	id, recipient_id FK, type, payload JSONB, read_at nullable, created_at
audit_events	id, actor_id FK, entity_type, entity_id, action, changes JSONB, created_at

• Создай SQLAlchemy 2 models, Alembic migration, foreign keys, UNIQUE, NOT NULL и нужные индексы.

• Статусы задачи: TODO, IN_PROGRESS, BLOCKED, DONE. Приоритеты: LOW, MEDIUM, HIGH, URGENT.

• Создай seed-команду: 2 workspaces, 4 users, 3 projects и 20 tasks. Не используй реальные пароли.

• Подумай о правилах удаления: нельзя оставлять «осиротевшие» комментарии и файлы. Документируй cascade/restrict.

Тестовая база данных

• Подними отдельную тестовую PostgreSQL (контейнер в Compose и service в CI). Схема создаётся теми же миграциями; каждый тест работает в транзакции с откатом.

• Напиши тесты на ограничения: повторный email и повторный membership отклоняются базой данных.

Критерий готовности: Чистая PostgreSQL создаётся через миграции; seed-данные загружаются повторяемо; ограничения БД реально работают и проверены тестами в CI.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: RDBMS, PostgreSQL, primary/foreign key, index, JOIN, transaction, ACID, ORM, SQLAlchemy, Alembic, migration, UUID, JSONB.

Контрольные вопросы: Что такое ORM? Почему UNIQUE лучше проверять не только в Python? Чем миграция отличается от create_all()?

ЭТАП 05

REST API и стандарты контрактов

Цель: Реализовать понятный версионированный HTTP API.

Метод и путь	Поведение
POST /api/v1/auth/register	Создать пользователя
POST /api/v1/auth/login	Получить access/refresh tokens
POST /api/v1/auth/refresh	Обновить access token
POST /api/v1/auth/logout	Отозвать refresh token
GET /api/v1/users/me	Текущий пользователь
GET, POST /api/v1/workspaces	Список (с ролью текущего пользователя) и создание
GET, POST /api/v1/workspaces/{id}/members	Список участников; добавить зарегистрированного пользователя по email
GET, POST /api/v1/projects	Список и создание (workspace_id)
GET /api/v1/projects/{id}	Получить проект
GET, POST /api/v1/tasks	Список и создание
GET, PATCH, DELETE /api/v1/tasks/{id}	CRUD задачи
GET /api/v1/tasks/{id}/history	История изменений (audit_events)
GET, POST /api/v1/tasks/{id}/comments	Комментарии
GET, POST /api/v1/tasks/{id}/attachments	Список и загрузка вложений
GET /api/v1/attachments/{id}/download	Скачать: поток или redirect на presigned URL
DELETE /api/v1/attachments/{id}	Удалить вложение и объект в MinIO
GET /api/v1/notifications	Уведомления
PATCH /api/v1/notifications/{id}/read	Отметить прочитанным
GET /api/v1/projects/{id}/stats	Статистика: всего, по статусам, по приоритетам, просрочено

Требования к контрактам

• Используй коды 200/201/204/400/401/403/404/409/422/429/500 по смыслу. Ошибки возвращай в едином JSON-формате с error.code, error.message, request_id.

• GET /tasks: фильтры project_id, status, priority, assignee_id, due_before; сортировка по created_at/due_at; page/page_size с ограничением размера страницы.

• Создай OpenAPI/Swagger документацию, минимум по одному request/response примеру для каждого типа endpoint.

• Заранее реши, кто может удалять задачи и вложения, и внеси это в матрицу прав (этап 07).

Критерий готовности: Все endpoints задокументированы; HTTP статусы корректны; ручные запросы воспроизводимы через curl/Postman.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: HTTP methods, REST, status codes, idempotency, query/path params, pagination, OpenAPI, Swagger, CORS.

Контрольные вопросы: Чем PATCH отличается от PUT? Почему GET не должен изменять данные?

ЭТАП 06

Pydantic и обработка ошибок

Цель: Не пропускать неверные данные в бизнес-логику.

• Создай отдельные Pydantic-схемы для CreateTask, UpdateTask, TaskResponse, RegisterUser и других контрактов.

• Валидируй email, минимальную длину пароля (например, 8 символов), длины строк, enum, допустимые значения pagination и корректность дат. Не разрешай менять системные поля через API.

• Настрой централизованный error handler. Не показывай клиенту stack trace, SQL и секреты.

• Напиши тесты на пустой title, невалидный email, отрицательный page_size, неизвестный status, лишние поля.

Критерий готовности: Невалидные запросы дают предсказуемые ошибки; внутренние исключения не раскрывают детали сервера.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Pydantic BaseModel, field validator, serialization, validation vs business rules, DTO, exception handler.

Контрольные вопросы: Чем Pydantic-схема отличается от SQLAlchemy-модели?

ЭТАП 07

Аутентификация и права доступа

Цель: Обеспечить безопасную работу нескольких команд.

• Хешируй пароли Argon2id или bcrypt; никогда не храни plaintext. Реализуй login, refresh-token flow, expiration и POST /auth/logout с отзывом refresh token; стратегию revocation выбери и объясни.

• Роли: OWNER управляет workspace и участниками; ADMIN управляет проектами и задачами; MEMBER работает с разрешёнными задачами. Зафиксируй матрицу прав. Создатель workspace становится OWNER; добавлять участников может только OWNER, роль нового участника — ADMIN или MEMBER.

• Каждый запрос к project/task/comment проверяет membership в workspace через сервер. Проверяй не только роль, но и принадлежность объекта.

• Тест: участник Workspace A не может читать/менять Task из Workspace B, даже зная UUID.

Критерий готовности: Тесты на 401/403/404 и попытки доступа к чужим объектам проходят.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Authentication vs authorization, JWT, access token, refresh token, RBAC, IDOR/BOLA, password hashing, OWASP.

Контрольные вопросы: Почему «скрыть кнопку» на frontend недостаточно для безопасности?

ЭТАП 08

Бизнес-логика задач и SQL

Цель: Реализовать корректное поведение при реальных сценариях.

• Создание задачи требует доступ к проекту; назначенный исполнитель должен состоять в workspace; due_at хранится в UTC.

• При смене status/assignee/priority сохраняй audit_event со старым и новым значением. Отдавай историю через GET /tasks/{id}/history в хронологическом порядке.

• Список задач не должен создавать N+1 queries. Добавь индексы по project_id, status, assignee_id, due_at и проверь план запроса.

• Обновление нескольких связанных записей выполняй в одной транзакции. При ошибке делай rollback.

• Просроченной считается задача с due_at в прошлом и статусом не DONE. Это правило используют доска задач и статистика проекта.

Критерий готовности: Есть тесты бизнес-правил, EXPLAIN ANALYZE для списка задач и пример отката транзакции.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: N+1 query, eager loading, EXPLAIN ANALYZE, B-tree index, transaction isolation, rollback, UTC, timezone.

Контрольные вопросы: Почему 100 SQL-запросов могут быть хуже одного JOIN?

ЭТАП 09

Redis: cache и rate limiting

Цель: Ускорить чтение и ограничить злоупотребления.

• Кэшируй GET /projects/{id}/stats в Redis на 60 секунд с ключом, учитывающим project_id и контекст доступа.

• При создании, удалении или изменении задачи инвалидируй соответствующую статистику.

• Добавь rate limit для /auth/login. При недоступности Redis реши, что должно происходить с обычными read endpoints и с защитными лимитами.

• Измерь время запроса с кэшем и без него, опиши результаты.

Критерий готовности: Повторный запрос использует cache; изменения отражаются без устаревших результатов дольше допустимого TTL.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Redis, cache-aside, TTL, cache invalidation, rate limiting, fixed/sliding window, cache stampede.

Контрольные вопросы: Почему нельзя кэшировать приватные данные под общим ключом?

ЭТАП 10

Celery: фоновые задачи

Цель: Вынести медленные операции из HTTP-запросов.

• Создай Celery app и отдельный worker container. Используй Redis broker; настрой retry/backoff и time limits.

• При назначении исполнителя создай запись в notifications (type task_assigned) и поставь в очередь задачу отправки письма. Для dev используй Mailpit или аналог, чтобы письма не уходили реальным людям.

• Логируй task_id, количество попыток, результат; не отправляй письмо внутри транзакции HTTP handler.

• Advanced: transactional outbox, чтобы избежать потери события между commit в БД и отправкой в брокер.

Критерий готовности: API отвечает быстро; worker независимо обрабатывает задания; сбой почты не ломает создание задачи.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Message broker, Celery, worker, queue, retry, exponential backoff, at-least-once delivery, idempotency, transactional outbox.

Контрольные вопросы: Что будет, если worker обработает одно сообщение дважды?

ЭТАП 11

Cron и Celery Beat

Цель: Автоматизировать регулярные процессы.

• Ежедневно в 09:00 UTC находи задачи с дедлайном в ближайшие 24 часа и создавай исполнителям уведомления (type deadline_reminder).

• Раз в неделю создавай digest для владельца workspace (type weekly_digest): сколько задач создано, завершено и просрочено.

• Раз в сутки очищай старые временные файлы или истёкшие записи (с retention policy).

• Проверь повторный запуск scheduler: не должно быть двух независимых Beat-инстансов, случайно выполняющих один и тот же график.

Критерий готовности: Периодические задачи запускаются по расписанию, безопасны при повторном выполнении, видны в логах.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Cron expression, Celery Beat, scheduler, UTC vs local time, idempotency, distributed lock, retention.

Контрольные вопросы: Почему повторная отправка напоминания может быть проблемой?

ЭТАП 12

Загрузка файлов и MinIO

Цель: Хранить вложения отдельно от базы данных.

• Подними MinIO в Compose; создай bucket и сервис загрузки. В БД сохраняй только метаданные и object_key.

• Ограничь размер (например, 10 MB), допустимые MIME-типы (например, PNG, JPG, PDF, TXT) и права доступа. Не доверяй имени файла от клиента.

• Реализуй GET /tasks/{id}/attachments (список) и GET /attachments/{id}/download: авторизованная отдача файла или redirect на короткоживущий presigned URL.

• При удалении вложения через DELETE /attachments/{id} обеспечь согласованность записи БД и объекта; опиши сценарии ошибок.

Критерий готовности: Пользователь не может скачать вложение из чужого workspace; файл переживает перезапуск контейнера.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: S3, object storage, bucket, presigned URL, MIME type, multipart upload, volume, eventual consistency.

Контрольные вопросы: Почему файлы лучше не хранить в файловой системе ephemeral контейнера?

ЭТАП 13

Отдельный React frontend

Цель: Сделать реальный клиент, работающий только через API.

• Создай React + TypeScript + Vite app отдельно от Flask. Экраны делай по макетам «TaskFlow — UI дизайн»: pixel-perfect не требуется, важны состояния загрузки, ошибок и пустых данных.

• Обязательный минимум — экраны основного сценария: Login, Register, Workspaces, Members, Projects, Task Board с диалогом создания задачи, Task Details. Notifications и Stats — после минимума, иначе Advanced.

• Можно взять библиотеку компонентов, чтобы не тратить время на вёрстку; выбор объясни в README.

• Настрой API client, базовый URL из environment config, обработку 401 и ошибок. Не подключай frontend напрямую к БД/Redis.

• Реализуй формы, фильтры задач, статусы загрузки, сообщения об ошибках и простую навигацию.

• Для development настрой CORS на разрешённый origin; для production предпочтителен единый домен с reverse proxy и /api.

Критерий готовности: Пользователь проходит основной сценарий через браузер без Postman: регистрация → workspace → участник → проект → задачи → комментарий и вложение; frontend и backend собираются отдельно.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: SPA, React, TypeScript, Vite, fetch/Axios, CORS, same-origin policy, access token storage, reverse proxy.

Контрольные вопросы: Почему CORS — не механизм авторизации?

ЭТАП 14

Тестирование и качество кода

Цель: Превратить тесты в систему, которая ловит регрессии.

• Базовая инфраструктура уже есть с этапов 03–04. Разложи тесты по уровням: unit — services, validation, permission helpers; integration — PostgreSQL, Redis, Celery (где нужно); API — Flask test client.

• Используй factories и контролируемые тестовые данные вместо ручных INSERT. Не запускай тесты на production DB.

• Добавь type checking (mypy/pyright) и coverage report в CI. Цель: минимум 75% по backend; важные права доступа покрыть полностью по сценариям.

• Напиши негативные тесты: чужой workspace, duplicate membership, повторный запрос, несуществующий объект, сбой worker.

Критерий готовности: Одна команда запускает тесты; проверки воспроизводимы на чистой машине и в CI.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Unit vs integration vs end-to-end, pytest fixture, mocking, test isolation, coverage, CI, static analysis.

Контрольные вопросы: Почему 100% coverage не означает отсутствие багов?

ЭТАП 15

Логи, метрики и мониторинг

Цель: Научиться находить ошибки без доступа к debugger.

• Structured JSON logs: timestamp, level, service, request_id, method, path, status_code, duration_ms; исключи пароли и токены.

• Prometheus metrics: HTTP requests total, latency histogram, error count, worker task failures, queue processing duration. Используй /metrics с контролируемым доступом.

• Подними Prometheus и Grafana. Добавь dashboards: RPS, p95 latency, 5xx rate, worker errors.

• Подключи Sentry (опционально hosted). Настрой минимум два алерта: высокий процент 5xx и недоступность API.

• Проведи эксперимент: искусственно замедли endpoint, найди проблему по графикам и логам.

• Advanced: Loki и log shipper (например, Grafana Alloy), чтобы искать логи по request_id прямо в Grafana.

Критерий готовности: В Grafana видны метрики; по request_id запрос находится в логах; ошибка создаёт событие в Sentry.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Observability, logs vs metrics vs traces, Prometheus, Grafana, Loki, p95/p99, histogram, RED metrics, Sentry, OpenTelemetry.

Контрольные вопросы: Чем 500 errors отличается от высокого p95 latency?

ЭТАП 16

Безопасность и конфигурация

Цель: Избежать типичных ошибок перед публикацией приложения.

• Конфигурация через env vars и .env.example; реальные секреты только в окружении сервера/secret store. В production отключи debug.

• Настрой CORS, HTTPS, security headers, ограничение размера тела запроса, безопасную обработку файлов, rate limits.

• Контейнеры не должны запускаться root без причины. PostgreSQL/Redis/MinIO не публикуй в открытый интернет.

• Проверь dependency vulnerabilities, добавь backup/restore план для PostgreSQL и persistent storage.

Критерий готовности: В репозитории нет секретов; наружу доступны только необходимые HTTP(S) порты; восстановление БД описано.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: OWASP Top 10, secrets management, least privilege, HTTPS/TLS, reverse proxy, backup, RPO/RTO, supply chain security.

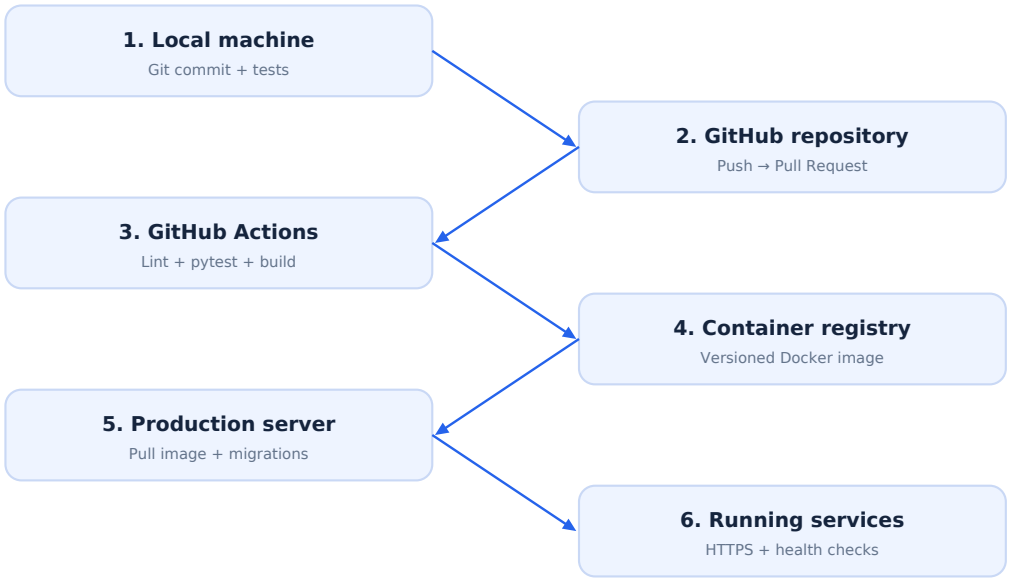
Контрольные вопросы: Почему Docker Compose не делает приложение автоматически безопасным?

ЭТАП 17

CI/CD: путь от ноутбука до registry

Цель: Автоматизировать проверки и сборку каждого изменения.

Схема поставки кода



• Git workflow: feat/* branch → PR → review → merge в main. Запрет merge при красных проверках действует с этапа 03 — проверь настройки protected branch.

• Расширь CI на PR: lint и type check, тесты с PostgreSQL service и coverage, build Docker image.

• На merge в main: build immutable Docker image с tag по commit SHA; push в GHCR (или другой registry).

• Не используй latest как единственный способ идентифицировать релиз. Обеспечь воспроизводимую версию image.

• Секреты registry и SSH/deploy keys храни в GitHub Secrets/Environments, а не в workflow YAML.

Критерий готовности: PR с падающим тестом не проходит checks; успешный merge публикует версионированный image.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: CI vs CD, GitHub Actions, workflow, runner, Docker registry, GHCR, artifact, commit SHA, protected branch, environment secrets.

Контрольные вопросы: Чем docker build отличается от docker push? Как откатить плохой image?

ЭТАП 18

Production deployment на VPS

Цель: Опубликовать приложение с доменом, HTTPS и управляемыми релизами.

• Выбери Linux VPS (например, Ubuntu), установи Docker/Compose, настрой SSH keys, firewall и отдельного deploy user.

• Подними reverse proxy (Nginx/Caddy), HTTPS с автоматическим обновлением сертификатов и доменом или поддоменом.

• Используй compose.prod.yaml: без hot reload, без debug, с persistent volumes, restart policies и health checks.

• Пайплайн деплоя: скачать image по SHA → выполнить миграции как отдельный one-off job → обновить сервисы → проверить /health и smoke tests.

• Опиши rollback: возврат предыдущего image; учти, что несовместимые миграции БД нельзя автоматически «откатить» только заменой image.

• Настрой backups, ограничь доступ к admin endpoints, протестируй перезапуск VPS/контейнеров.

Критерий готовности: С публичного HTTPS URL можно открыть frontend и работать с API; deployment повторяем, есть инструкции rollback и backup.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: VPS, Linux, SSH, firewall, DNS, TLS, Nginx/Caddy, health check, zero downtime, database migration strategy, rollback.

Контрольные вопросы: Что произойдёт при падении сервера во время миграции?

ЭТАП 19

Финальная приёмка проекта

Цель: Продемонстрировать систему и подтвердить выполнение требований.

End-to-end сценарий

- 1. Зарегистрировать двух пользователей и создать workspace.
- 2. Добавить второго пользователя в workspace с ролью MEMBER.
- 3. Создать проект и три задачи, назначить исполнителя и дедлайны.
- 4. Изменить статус задачи и увидеть запись в истории изменений (audit event).
- 5. Добавить комментарий, загрузить вложение в MinIO и скачать его.
- 6. Убедиться, что Celery отправил уведомление и cron сформировал напоминание.
- 7. Попробовать получить чужую задачу и получить отказ.
- 8. Найти запрос по request_id в логах, открыть метрики в Grafana и событие ошибки в Sentry.
- 9. Запустить CI на PR, выполнить deploy и показать production URL.

Что предоставить на проверку

Артефакт	Требование
GitHub repo	README, код, миграции, тесты, workflow, .env.example
Architecture.md	Диаграмма, объяснение компонентов и trade-offs
OpenAPI	Актуальная спецификация endpoint и ошибок
Frontend	Обязательные экраны по макетам, отдельная сборка
Demo	Видео или живой показ полного сценария
Observability	Dashboard + примеры логов и алертов
Production	HTTPS URL, smoke tests, rollback и backup инструкции

Критерий готовности: Все обязательные сценарии проходят, проект можно поднять с нуля по README, продемонстрировать и объяснить.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Acceptance criteria, Definition of Done, README, runbook, smoke test, postmortem, technical debt.

Контрольные вопросы: Сможет ли другой разработчик поддерживать приложение без устных объяснений?

ЭТАП 20

Advanced: инженерные вызовы

Цель: После стабильного MVP приблизить систему к реальным production-проблемам.

• Optimistic locking: два пользователя меняют одну задачу одновременно. Добавь version и возвращай 409 при конфликте.

• Idempotency key: повторный POST из-за network retry не создаёт дубликат.

• Transactional outbox: изменение задачи и запись события атомарны; отдельный dispatcher доставляет события в очередь.

• Tracing: добавь OpenTelemetry trace для API → PostgreSQL → Celery, найди bottleneck.

• Load testing: сгенерируй 100k tasks, проведи k6/Locust тест и сравни p95 до и после индексов.

• Realtime: реализуй SSE/WebSocket уведомления, объясни reconnect и fallback.

• Приглашения по email: таблица invitations с токеном и сроком действия, письмо через Celery, принятие приглашения при регистрации.

• Централизованные логи: Loki и Grafana Alloy, поиск по request_id в Grafana.

Критерий готовности: Для каждого advanced задания есть воспроизводимый тест, метрика результата и краткое архитектурное объяснение.

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Optimistic concurrency, idempotency key, transactional outbox, distributed tracing, load testing, SSE, WebSocket, invitation token.

Контрольные вопросы: Какие проблемы решают эти подходы и почему они не обязательны для первого MVP?

ЭТАП 21

Как проходит code review

Цель: Учиться аргументировать инженерные решения, а не только писать код.

• Правила действуют с первого Pull Request (этап 02), а не только в конце проекта.

• Каждый PR должен включать: контекст задачи, что изменилось, как проверить, какие тесты добавлены, риски и скриншоты при изменении UI.

• На review объясни: почему выбрал такой слой, как обработал ошибки, что будет при повторном запросе и при недоступности внешнего сервиса.

• Не объединяй десятки несвязанных изменений в одном PR. Делай небольшие проверяемые шаги.

• Для каждого этапа записывай 3–5 новых терминов, их определения и один пример применения в TaskFlow.

Контрольный лист перед merge

[ ] Поведение соответствует acceptance criteria

[ ] Добавлены тесты на happy path и ошибки

[ ] Нет секретов и чувствительных данных в логах

[ ] Есть миграция при изменении схемы

[ ] API-контракт и README обновлены

[ ] CI зелёный; код понятен другому разработчику

САМОСТОЯТЕЛЬНО ИЗУЧИТЬ

Погугли и объясни своими словами: Code review, clean code, SOLID (без фанатизма), refactoring, backward compatibility, technical documentation.

Контрольные вопросы: Сможешь объяснить, почему твоё решение проще и надёжнее альтернатив?

Главный принцип: не подключай технологию только ради строки в резюме. Сначала сформулируй проблему, затем докажи, что выбранный инструмент действительно помогает её решить.